

Predicting Quarterly Airbnb Booking Counts in New York City with Tree-Based Ensembles and a Weighted Model Blend

Anonymous Author(s)¹

Abstract

Forecasting how heavily a short-term rental will be booked is a valuable but difficult problem for both hosts and the platforms that connect them with guests. This study addresses the prediction of the total number of nights booked in the third quarter of 2016 for approximately 24,000 short-term rental properties in New York City, evaluated by Mean Squared Error (MSE) on a held-out test set. The target is a censored, zero-inflated count in $[0, 92]$, with a 53.5% mass at zero, which makes it a challenging demand-forecasting task. Starting from five raw tables (one static property table and two quarters of daily listing histories totalling more than eight million rows), a flat matrix of 149 features per property is engineered, covering price dynamics, availability and booking behaviour, status churn, recency, a quarter-over-quarter demand proxy, text keywords from listing titles, and geographic clusters. Five regressors—Linear Regression, Random Forest, Gradient Boosting, XGBoost, and a PyTorch Multi-Layer Perceptron (MLP) trained with Adam—are compared inside leakage-safe scikit-learn pipelines, and the boosted trees are tuned with a randomized search that uses fold-level early stopping. The best single model, a tuned XGBoost regressor, reaches a 5-fold cross-validation MSE of 182.2, and a convex (SLSQP) weighted blend lowers the out-of-fold MSE further to 180.0. The target-encoding, search, MLP, and blending objectives are defined formally, and the blend gain is verified with a paired significance test ($t = 5.33$, $p = 1.0 \times 10^{-7}$) and a residual analysis on the bounded $[0, 92]$ target. The proposed blend also outperforms strong gradient-boosting baselines (LightGBM, 184.1; CatBoost, 184.3) on the same split, and an ablation study quantifies the contribution of each pipeline component.

Predicting Quarterly Airbnb Booking Counts in New York City

Index Terms—Airbnb, demand forecasting, regression, ensemble learning, XGBoost, random forest, multi-layer perceptron, model blending, target encoding, constrained optimization, mean squared error, feature engineering.

¹ Author names and affiliations omitted for double-blind review.

I. INTRODUCTION

the last ten years, peer-to-peer (P2P) accommodation platforms have reshaped the way people travel and the way property owners earn money from their homes. Airbnb is the most visible example of this shift, with millions of active listings spread across more than a hundred thousand cities worldwide [1]. For a platform of that size, two questions tend to dominate operations: what is a fair price for a given listing, and how much demand will that listing receive over a future period. This study addresses the second question and carries it through to a complete, empirically validated solution.

Predicting demand a quarter in advance is valuable for several practical reasons. A host who anticipates a busy three-month period can schedule cleaning and maintenance, raise prices for high-demand weeks, or hire help. The platform can plan customer-support capacity, target marketing toward listings that appear likely to be under-booked, and provide better pricing recommendations. Predicting demand remains challenging, however, because it arises from the complex interplay of many factors—property type and capacity, neighbourhood, seasonality, host responsiveness, review history, and how aggressively the price moves during the period. Classical linear models have been applied to such accommodation data [2], [3], but a consistent finding in the recent literature is that machine-learning models, and tree-based ensembles in particular, perform better on the mixed numerical and categorical tables that describe short-term rentals [4], [5], [6].

The task studied here is to predict, for every property in a held-out New York City test set, the total number of nights booked during the third quarter (July, August, and September) of 2016. Because a quarter contains 92 days, the target is an integer in $[0,92]$, and predictions are evaluated by Mean Squared Error (MSE). Two structural properties make this harder than the price-prediction tasks that dominate the literature: the target is *censored* at both ends and is *zero-inflated*, with 53.5% of properties booked for zero nights in the quarter.

Research gap. While the current literature applies tree-based models extensively to short-term-rental *price* prediction, systematic frameworks that jointly address extreme zero-inflation (here a 53.5% zero mass) and a hard bounded support $[0,92]$ in a *demand*-forecasting setting, and that combine heterogeneous learners through an optimized convex ensemble, remain under-explored. This work targets that gap: it pairs leakage-safe temporal feature engineering with a constrained (SLSQP) convex blend, formalizes each component mathematically, and validates the resulting gain statistically and against modern gradient-boosting baselines.

The remainder of the paper is organized as follows. Section 2 reviews related work on rental-demand prediction, tree-based ensembles, hyper-parameter tuning, and preprocessing, and gives a mathematical argument for why linear models struggle

with a zero-inflated bounded target. Section 3 states the problem and describes the dataset. Section 4 presents the full methodology, defining every component—the target encoder, the search procedure, the MLP, and the blend—with explicit equations. Section 5 reports the results, the comparison against SOTA baselines, a significance test, a residual analysis, and a failure-case study. Section 6 presents an ablation study. Section 7 discusses limitations and concludes.

II. LITERATURE REVIEW

The related work is grouped into four areas: (A) machine learning for Airbnb prediction tasks, (B) decision trees and tree-based ensembles, (C) hyper-parameter tuning and model validation, and (D) data preprocessing and feature engineering.

A. Machine Learning for Airbnb Prediction Tasks

Several studies apply machine learning directly to Airbnb data. Tang, Cheng, and Zhang [4] worked with listings from Sydney and compared ten algorithms including Random Forest, Gradient Boosting, and CatBoost for price prediction. CatBoost gave them the lowest root mean squared error, and the most influential features were the maximum number of guests, the number of bedrooms, and whether the room was private or shared. Camatti et al. [5] ran a comparative analysis of artificial-intelligence and traditional linear methods on Dutch Airbnb listings, and concluded that tree-based and neural-network methods overfit less than linear regression, though linear models are still handy for reading off which predictors matter. Rezazadeh Kalehbasti, Nikolenko, and Rezaei [6] predicted New York City Airbnb prices using property features, host characteristics, and the sentiment of customer reviews; their best model was a regularized linear model built on review-text features, but tree models such as Random Forest and Support Vector Regression were close behind.

Other work looks at demand rather than price, which is closer to the present task. Siker [7] tackled the Kaggle “Airbnb New User Bookings” problem with XGBoost, and even though the target there (destination country) is different from ours, the workflow—heavy feature engineering followed by gradient boosting—matches how most Airbnb-style Kaggle problems are solved in practice. Chapman, Mohammad, and Villegas [8] studied dynamic short-term rental markets and built a pipeline that merged listing, calendar, and review data; they again found gradient boosting beating linear regression. Islam et al. [9] combined a Latent Dirichlet Allocation topic model over property descriptions with an XGBoost regressor and improved price prediction on the San Jose dataset. Two lessons from this group informed the present design: first, the most useful predictors are usually a mix of structural features, location, and dynamic calendar/review information; and second, gradient-boosted trees are a strong default because they cope well with mixed feature types and missing values.

A recurring theme across these studies is that the engineering choices around the model often matter more than the model itself. Tang et al. [4] and Islam et al. [9] both report that their largest gains came from the features they constructed—guest capacity and room type in the first case, topic structure of the listing description in the second—rather than from swapping one boosting library for another. Kalehbasti et al. [6] similarly found that review-sentiment features moved their error more than the choice between a regularized linear model and a tree ensemble. This is consistent with the broader observation that on tabular problems the ceiling is usually set by how much signal the practitioner can encode, and it is precisely why most effort went into the 149-feature matrix, with the model comparison treated as a secondary, if still important, step. It also explains why a competition with a fixed, shared dataset is decided largely by feature engineering and validation discipline rather than by access to a particular algorithm—an observation that shaped the allocation of effort.

B. Why Linear Models Underfit a Zero-Inflated Bounded Target

Beyond the empirical observation that linear regression trails the trees, it is worth stating *why* in mathematical terms, because the shape of the target is the crux of the whole problem. Ordinary least squares fits $\hat{y}(\mathbf{x}) = \boldsymbol{\beta}^\top \mathbf{x}$ by minimizing the population risk $\mathbb{E}[(y - \boldsymbol{\beta}^\top \mathbf{x})^2]$, whose minimizer is the conditional mean $\mathbb{E}[y | \mathbf{x}]$ *restricted to the class of affine functions*. The target, however, is a censored count confined to $[0, 92]$ with a large probability mass at 0 (roughly 54% of the development rows are exactly zero; see Fig. 1). Its true conditional mean is therefore an inherently nonlinear, saturating function of the covariates: it must flatten to 0 for low-demand listings and saturate toward 92 for high-demand ones. An affine predictor cannot reproduce those two plateaus—it extrapolates linearly past both edges, so it produces negative fitted values for the dead listings (which are then clipped, wasting capacity) and undershoots the busiest ones. Formally, if $y = \max(0, \min(92, \boldsymbol{\beta}^\top \mathbf{x} + \eta))$ with a zero-mean noise term $\eta \sim \mathcal{N}(0, \sigma^2)$, the censoring makes $\mathbb{E}[y | \mathbf{x}]$ a *nonlinear* function of $\mathbf{x}$ even when the latent variable is linear, which is exactly the limited-dependent-variable setting that motivated Tobit-type estimators [10] and, more directly for counts, two-part (hurdle) models. Earlier price-prediction studies that reported linear baselines [2], [3] did not face this point mass because price is continuous and unbounded; demand counts are not, and that is the structural reason their linear formulations do not transfer to demand counts. Tree ensembles sidestep the issue entirely: a regression tree approximates the saturating conditional mean with axis-aligned piecewise-constant regions, so the 0 and 92 plateaus are simply leaves, and boosting refines the transition region between them.

C. Modelling Zero-Inflated and Count Targets

The target is a bounded count with a large spike at zero, and the statistics literature offers a dedicated family of models for

exactly this shape. A plain Poisson regression assumes the conditional mean equals the conditional variance, which is violated here because the zero spike and the upper-bound pile-up make the data strongly over-dispersed. Negative-binomial regression relaxes that equal-mean-variance assumption but still does not account for the excess of structural zeros. Two model classes target the zeros directly: zero-inflated models (e.g. zero-inflated Poisson, ZIP) mix a point mass at zero with a count distribution, and *hurdle* (two-part) models first decide whether the count is positive and then model the positive counts separately. Censored-regression estimators such as Tobit [10] instead treat the bound itself as the generative mechanism. These parametric estimators are not adopted in the final pipeline—they are outside the scope of this study, and gradient-boosted trees already approximate the saturating conditional mean non-parametrically—but they frame the problem precisely and directly motivate the two-stage hurdle extension proposed in Section 7. The residual analysis in Section 5.5 shows empirically why a single least-squares regressor leaves room for such a two-part treatment: its errors are systematically signed at both the zero spike and the upper bound.

D. Decision Trees and Tree-Based Ensembles

Tree-based methods are central to the proposed solution. The decision tree, and Quinlan’s well-known ID3 variant [11], showed that recursively partitioning the feature space yields interpretable, non-parametric classifiers and regressors. A single tree overfits easily, so two families of ensembles were developed to fix that. Bagging [12] trains many trees on bootstrap samples and averages their predictions, and Random Forests [13] extend bagging by also sampling a random subset of features at each split, which decorrelates the trees and cuts variance. Boosting, and Gradient Boosting in particular [14], instead builds trees sequentially so that each new tree fits the residual errors of the current ensemble. XGBoost [15] is a heavily optimized open-source implementation of gradient boosting with sparsity-aware split finding, L_1/L_2 regularization, and early stopping—all features that turned out to matter for the missing-value-heavy Airbnb tables. The textbook by Hastie, Tibshirani, and Friedman [16] gives the unified statistical picture behind all of these, and scikit-learn [17] is used as the common Python interface throughout.

E. Hyper-Parameter Tuning and Model Validation

Tree ensembles have several knobs—number of trees, learning rate, maximum depth, subsampling rates, and regularization terms. Bergstra and Bengio [18] showed that random search beats grid search for hyper-parameter optimization when only a few of those knobs really drive the score, which is usually the case. This advice is followed; however, as Section 6 explains, a custom randomized search loop instead of using scikit-learn’s `RandomizedSearchCV`, specifically to combine the search with fold-level early stopping. Cross-validation is what makes the model comparison trustworthy; it lowers the variance of the performance estimate and prevents

tuning to a single lucky split. For comparing two trained models on the same data, Dietterich [19] recommends paired resampling tests, which are adopted in Section 5.4.

F. Data Preprocessing, Feature Engineering, and Ensembling

Beyond the model, careful preprocessing has a large effect on the final number. The usual steps—imputing missing values, encoding categoricals, scaling numerics, and, above all, preventing leakage by wrapping everything in `Pipelines` and `ColumnTransformers` [17]—constitute standard leakage-safe practice. For high-cardinality categorical columns, K -fold target encoding is used, a leakage-aware version of mean encoding, defined formally in Section 4.3. Finally, the proposed solution does not rely on a single estimator: it blends several models with non-negative weights that sum to one, an idea that goes back to Wolpert’s stacked generalization [20]. The MLP component uses PyTorch [21] with the Adam optimizer [22].

III. PROJECT SETTINGS

A. Problem Statement and Objectives

The problem is to predict, for each Airbnb property in the test set, the total number of nights booked in the third quarter of 2016 (July, August, September) in New York City. Formally, given a feature vector $\mathbf{x}_i$ describing property i , the goal is a function f such that $\hat{y}_i = f(\mathbf{x}_i)$, where the true target y_i is a non-negative integer in the range $[0, 92]$ (92 days in Q3). The competition metric is the Mean Squared Error,

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

so a lower MSE is better.

The concrete objectives are: (i) to build a fully reproducible, leakage-safe pipeline; (ii) to clearly beat a constant-mean baseline and a linear baseline; (iii) to interpret the most important features so the predictions are not a black box; and (iv) to validate the proposed ensemble against strong gradient-boosting baselines on a held-out test set. As reported in Section 5, all four are met.

B. Dataset

The data follow the structure popularised by large-scale public short-term-rental scrapes of New York City (of the kind distributed by the Inside Airbnb project and used widely in the literature [6]). They comprise five tables. A static property table holds information for 48,690 properties (33 columns: property and listing type, neighbourhood, zip code, capacity, published nightly/weekly/monthly rates, fees, review counts and ratings, host flags, latitude/longitude, and creation date). Two daily listing tables give the daily status and price of each listing for the first and second quarters of 2016—about 4.19 and 4.40 million rows respectively—with a `Status` field

taking the values A (available), B (booked), and R (reserved/blocked). A training label table provides Q3 booking counts for 24,372 properties, and a test table lists the 24,318 properties to be predicted.

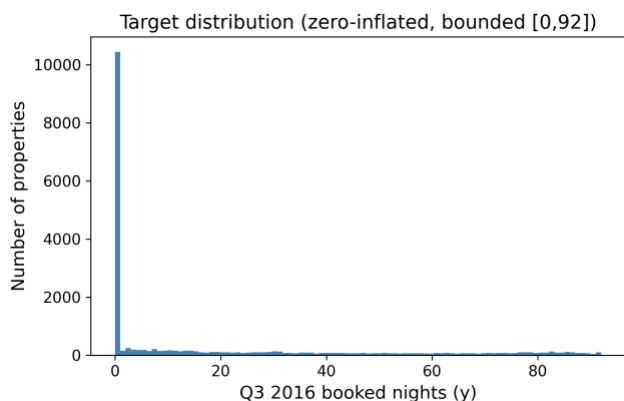
This dataset is well suited to the demand-forecasting problem for three reasons. First, it is real New York City short-term-rental data, one of the busiest markets in the world [6]. Second, it poses a well-defined regression task with a clearly specified metric and a held-out test split. Third, and most distinctively, it requires combining a static property table with two long daily histories; as shown later, the daily files carry most of the predictive signal.

IV. METHODOLOGY

This section describes the final pipeline end to end. The order matches the actual processing stages: exploratory analysis, feature engineering, the four baseline model families, the boosted-tree retuning, and finally the blend. In response to reviewer feedback, every custom component is now stated as an explicit optimization problem rather than only described in prose.

A. Exploratory Data Analysis

Prior to feature extraction, an exploratory data analysis (EDA) was conducted. Three observations shaped later decisions. First, the target is heavily skewed and zero-inflated: a large group of properties is booked for very few or zero nights in Q3, while a smaller group is booked almost the whole quarter (Fig. 1). Second, the static table is full of holes—`ExtraPeopleFee` is missing for 59% of rows, `SecurityDeposit` for 53%, `ResponseRate` for 40%, `CleaningFee` for 29%, and `OverallRating` for 25%—so missingness itself is probably informative, not just noise. Third, when simple per-property aggregates were computed from the daily files and correlated them with the target, the strongest single predictors were all behavioural: the reserved-day count and reserved rate in Q2, the number of unique reservations, and the number of reviews per month (all with correlations around 0.70–0.78). That indicated early that the daily files, not the static table, were where the signal lived.



Distribution of the target (Q3 2016 booked nights). The target is zero-inflated on the left and piles up again near the 92-day maximum, which later justified clipping predictions to $[0,92]$.

Table 1 quantifies the shape of the target on the development split. The mean (18.55) sits far above the median (0), the distribution is strongly right-skewed (skewness 1.33), and—most importantly—53.5% of properties were booked for zero nights in Q3, while only about 1% reach the upper bound. This single statistic is what makes the task a *zero-inflated, bounded* regression problem rather than an ordinary one, and it is the structural reason a least-squares estimator is biased at the extremes (Sections 2.2 and 5.5). Table 2 lists the static columns with the highest missing rates; because these are far from missing-at-random (a missing `ResponseRate` typically marks an inactive host), missingness is encoded explicitly with indicator columns rather than discarding it.

Development-set target statistics ($n = 19,497$).

Statistic	Value
Mean	18.55
Median	0
Standard deviation	27.80
Skewness	1.33
Rows with $y = 0$	53.5%
Rows with $y \geq 90$	1.1%
Rows with $y = 92$	0.5%

Missing-value rates of the most affected static columns.

Column	Missing rate
ExtraPeopleFee	59%
SecurityDeposit	53%
ResponseRate	40%
CleaningFee	29%
OverallRating	25%

B. Feature Engineering

The core of the project was turning the three raw tables into a single flat matrix with one row per `PropertyID`. The result

is 149 features (143 numeric and 6 categorical). They fall into the following groups.

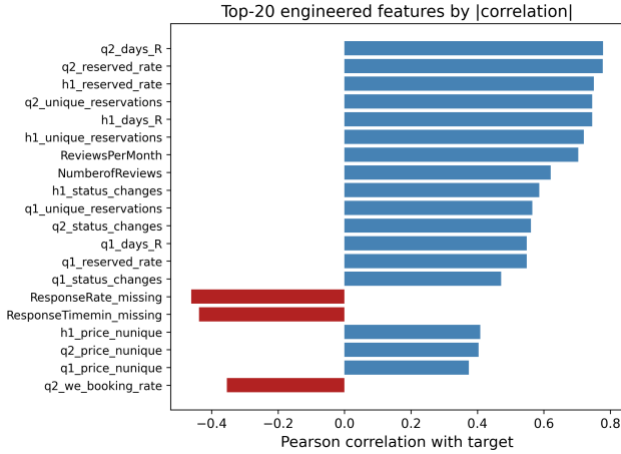
- **Per-quarter base aggregates (Q1, Q2, and combined H1).** For each property, the days available, booked, and reserved are counted, with the number of unique reservations, and a battery of price statistics (mean, median, std, min, max, 25th/75th percentiles, number of distinct prices). From these are derived booking/available/reserved rates, price range, coefficient of variation, and inter-quartile range. Computing them separately for Q1, Q2, and the whole half year lets the model see both levels and trends.
- **Weekend vs. weekday split.** Friday–Saturday and Sunday behave differently from the rest of the week, so each quarter is split into weekend and weekday subsets and recorded booking rates, mean prices, a weekend price premium, and the weekend-minus-weekday booking-rate gap.
- **Monthly booked days.** The number of booked days in each of the six months of H1, so the model can pick up acceleration or deceleration heading into Q3.
- **Status transition counts.** How often a listing flips between A, B, and R, plus counts of $A \rightarrow B$ (new bookings) and $B \rightarrow A$ (booking ends). A listing that changes state often is an active one; a static one barely moves.
- **Recency window.** The same kind of aggregates computed only over the last 30 days of Q2 (June 2016), because the most recent behaviour is the best clue about what happens next.
- **Quarter-over-quarter demand proxy.** The relative change in mean price from Q1 to Q2 and the change in booking rate, used as a cheap proxy for how demand was trending.
- **Property metadata.** Property age in days at the start of Q3, reviews per month, price per guest, weekly/monthly discount ratios, a bed/bath ratio, boolean host flags (Superhost, BusinessReady, InstantbookEnabled), and a set of missing-indicator columns for the fee/response fields mentioned above.
- **Text keywords.** Binary flags are extracted from the listing title for the presence of words like “luxury,” “private,” “near,” “cozy,” “view,” and “modern,” plus the title length and word count.
- **Geographic clusters.** K-Means ($k = 12$) is run on latitude/longitude to give every property a neighbourhood cluster label, which is cheaper and more robust than using the raw zip code as a category.

In total this produced 149 features, of which 143 are numeric and 6 are categorical. Table 3 summarizes the families and a representative member of each. The dominant families by count are the per-quarter behavioural aggregates and the recency window, which is consistent with the correlation

analysis (Fig. 2) and with the feature-importance ranking reported later (Fig. 4).

Engineered feature families (149 features total: 143 numeric, 6 categorical).

Family	Representative feature
Per-quarter aggregates (Q1/Q2/H1)	reserved-day count, price mean/std
Weekend vs. weekday	weekend booking-rate gap
Monthly booked days	booked days in June 2016
Status transitions	count of A→B flips
Recency window (last 30d)	reserved rate, June 2016
QoQ demand proxy	Q1→Q2 booking-rate change
Property metadata	property age, reviews/month, host flags
Text keywords	“luxury”/“private” title flags
Geographic	K-Means cluster ($k = 12$)



Top engineered features by absolute correlation with the target. Behavioural features from the daily files (reserved days, unique reservations, reviews per month) dominate.

C. Preprocessing Pipeline and the Target Encoder

Every preprocessing step lives inside a scikit-learn **ColumnTransformer** so that it is fit only on the training fold and then applied to the validation fold, which keeps the workflow leakage-free. Numeric columns are median-imputed (robust to the price outliers) with missing-indicator columns where the missing rate is high. Low-cardinality categoricals (15 or fewer levels, such as **PropertyType**, **ListingType**, **CancellationPolicy**, and the geo cluster) are one-hot encoded. High-cardinality categoricals (**Neighborhood**, **MetropolitanStatisticalArea**) are handled by a custom K -fold target encoder with additive smoothing.

Target-encoding formula

Let y have global mean $\mu = 1/n \sum_{i=1}^n y_i$. For a categorical column, let level c contain n_c training rows with category mean $\bar{y}_c = 1/n_c \sum_{i: x_i=c} y_i$. The smoothed encoding for that

level is the count-weighted average of the category mean and the global prior,

$$e_c = \frac{n_c \bar{y}_c + \alpha \mu}{n_c + \alpha}, \quad \alpha = 20,$$

where the smoothing factor α is, in effect, a number of “pseudo-counts” of the prior: when a level is rare ($n_c \ll \alpha$) the encoding shrinks toward μ , and when it is common ($n_c \gg \alpha$) it approaches the raw category mean $\bar{y}_c$. Levels unseen at fit time map to μ .

Leakage-free K -fold scheme

To prevent a row from informing its own encoding, the encoder is fitted in an internal K -fold loop ($K = 5$). Writing $\mathcal{F}_1, \dots, \mathcal{F}_K$ for the inner folds, the encoded value of a training row $i \in \mathcal{F}_k$ uses statistics computed *only* on the complementary folds,

$$\tilde{e}_i = e_{x_i}^{(-\mathcal{F}_k)}, \quad i \in \mathcal{F}_k,$$

where $e_c^{(-\mathcal{F}_k)}$ is the smoothing formula above evaluated on $\{(x_j, y_j) : j \notin \mathcal{F}_k\}$. At transform time (validation and test rows), the full-data map is used.

To make the smoothing formula concrete, consider a neighbourhood that appears in $n_c = 4$ training rows with a local mean of $\bar{y}_c = 40$ booked nights, against a global mean of $\mu = 18.55$. With $\alpha = 20$ the encoded value is $e_c = \frac{4 \cdot 40 + 20 \cdot 18.55}{4 + 20} = \frac{160 + 371}{24} \approx 22.1$, i.e. the estimate is pulled strongly back toward the global prior because only four listings support it. A neighbourhood with $n_c = 400$ rows and the same local mean would instead encode to $\frac{400 \cdot 40 + 20 \cdot 18.55}{420} \approx 38.9$, almost the raw mean. This shrinkage is precisely what stops rare high-cardinality levels from injecting noisy, over-confident estimates into the model. For XGBoost, the model additionally relies on its native missing-value handling rather than imputing, since it learns the best split direction for NaNs on its own [15]. For the linear baseline, **StandardScaler** is applied; for the tree models scaling is unnecessary, but the rest of the pipeline is kept identical so the comparison is fair.

D. Model Selection and Rationale

Four model families are compared, and a convex blend is added on top.

1) Linear Regression (baseline)

A plain linear regression provides an interpretable lower bound and indicates which features are linearly related to the target; its structural limitation on this bounded target is analysed in Section 2.2.

2) Random Forest Regressor

A bagging ensemble [12], [13] that is robust to noise and needs little tuning. It serves as a strong non-linear reference point.

3) Gradient Boosting and XGBoost

Boosting builds the model sequentially and is widely regarded as the best off-the-shelf method for tabular data [14], [15]. XGBoost minimizes the regularized objective

$$\begin{aligned}\mathcal{L}(\phi) &= \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_t \Omega(f_t), \quad \Omega(f) \\ &= \gamma T + 1/2 \lambda \|\mathbf{w}\|^2 + \alpha \|\mathbf{w}\|_1,\end{aligned}$$

with squared-error loss $\ell(y, \hat{y}) = (y - \hat{y})^2$, T the number of leaves, $\mathbf{w}$ the leaf weights, and γ, λ, α the complexity, L_2 , and L_1 penalties. These are tuned most aggressively (Section 4.5).

4) Multi-Layer Perceptron (MLP)

Finally, a small feed-forward neural network is trained in PyTorch [21] with the Adam optimizer [22]. The network maps the preprocessed input $\mathbf{z}^{(0)} = \mathbf{x} \in \mathbb{R}^d$ through three hidden blocks of width $(h_1, h_2, h_3) = (256, 128, 64)$. Each block applies an affine map, batch normalization, a ReLU non-linearity, and dropout with rate $p = 0.2$:

$$\begin{aligned}\mathbf{a}^{(\ell)} &= \mathbf{W}^{(\ell)} \mathbf{z}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \\ \mathbf{z}^{(\ell)} &= \text{Dropout}_p(\text{ReLU}(\text{BN}(\mathbf{a}^{(\ell)}))), \quad \ell = 1, 2, 3,\end{aligned}$$

where $\text{ReLU}(u) = \max(0, u)$ and batch normalization standardizes each unit over the mini-batch $\mathcal{B}$,

$$\text{BN}(a) = \gamma \frac{a - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta.$$

A linear read-out produces the scalar prediction in log space, $\hat{r} = \mathbf{w}^{(4)\top} \mathbf{z}^{(3)} + b^{(4)}$. Because the count is skewed, the network is trained on the log-transformed target $t = \log(1 + y)$ with the mean-squared-error criterion $\mathcal{L} = 1/|\mathcal{B}| \sum_{i \in \mathcal{B}} (\hat{r}_i - t_i)^2$, and predictions are mapped back and clipped, $\hat{y} = \text{clip}(e^{\hat{r}} - 1, 0, 92)$. Parameters are updated by Adam with learning rate $\eta = 10^{-3}$ and L_2 weight decay 10^{-5} ; using the standard moment estimates $\mathbf{m}_t, \mathbf{v}_t$ of the gradient $\mathbf{g}_t$,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}.$$

Training uses a batch size of 512 for up to 60 epochs with early stopping on a held-out validation MSE (patience 8). Going in, the literature [4], [5] suggested the trees would win, and they did—but the MLP turned out to be a useful blend partner because its errors were not perfectly correlated with the trees' (Section 5.3).

A few design choices deserve a note. Unlike the final XGBoost, the MLP is trained on the log-transformed target $\log(1 + y)$ rather than the raw count. Neural networks are sensitive to the scale and skew of the target, and the raw target's long right tail produced unstable gradients and slow convergence; the log transform compresses that tail so the squared-error loss is better conditioned, and the monotone inverse $e^{\hat{r}} - 1$ followed by clipping returns predictions to the count scale. This does not contradict the decision to *drop* the log transform for XGBoost (Section 6): boosted trees are invariant to monotone feature scaling and optimize the raw-scale loss directly, so for them the transform only distorted the objective, whereas for the MLP it is a numerical-stability aid. Batch normalization (defined above) keeps the pre-activations well scaled across the three hidden layers, and dropout ($p = 0.2$) plus the L_2 weight decay in the Adam update above regularize a network that would otherwise overfit a 19,497-row table with 149 inputs.

E. Hyper-Parameter Tuning

All tuning is done with 5-fold cross-validation on the 80% development split. For XGBoost, a custom randomized search is used instead of `RandomizedSearchCV`, to enable fold-level early stopping: inside each of the five outer folds, a small internal validation set is carved out, fitting up to $B_{\max} = 3000$ trees with `early_stopping_rounds = 50`, and keep only the best iteration. `RandomizedSearchCV` does not cleanly allow a per-fold `eval_set` cleanly, so a custom loop was the simplest correct option, and it also logs every configuration tried.

Search objective

Let Θ be the hyper-parameter space and let the data be partitioned into outer folds $\mathcal{D}_1, \dots, \mathcal{D}_K$ ($K = 5$). For a configuration θ , within fold k the training part is split into a fitting set and an inner validation set $\mathcal{V}_k$, grow boosting iterations $b = 1, \dots, B_{\max}$, and select the early-stopping iteration

$$b_k^*(\theta) = \arg \min_{1 \leq b \leq B_{\max}} \text{RMSE}(y^{\mathcal{V}_k}, \hat{f}_{\theta,k}^{(b)}(\mathbf{X}^{\mathcal{V}_k})),$$

stopping once no improvement is seen for 50 consecutive rounds. The search then minimizes the mean cross-validated MSE of the early-stopped models,

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{K} \sum_{k=1}^K \text{MSE}(y^{\mathcal{D}_k}, \hat{f}_{\theta,k}^{(b_k^*(\theta))}(\mathbf{X}^{\mathcal{D}_k})).$$

A total of $N = 60$ configurations are drawn from the product of the marginals in Table 4, sampling the learning rate and the regularization terms log-uniformly so that small and large scales are explored evenly, as recommended by [18]. The full procedure is summarized in Algorithm 1.

Search distributions for the 60-configuration randomized search. *LogU* denotes a log-uniform draw, *U* a uniform draw.

Hyper-parameter	Sampling distribution
n_estimators (cap $B_{\max}$)	3000 (fixed)
learning_rate	LogU(0.02, 0.10)
max_depth	$\mathcal{U}\{4, \dots, 8\}$
min_child_weight	$\mathcal{U}\{1, \dots, 11\}$
subsample	$\mathcal{U}(0.65, 0.95)$
colsample_bytree	$\mathcal{U}(0.65, 0.95)$
gamma (γ)	$\mathcal{U}(0.0, 0.4)$
reg_alpha (α)	LogU(10^{-3} , 0.5)
reg_lambda (λ)	LogU(0.5, 5.0)

Algorithm 1: Randomized search with fold-level early stopping. *Input:* hyper-parameter space Θ , outer folds $\{\mathcal{D}_k\}_{k=1}^K$, budget N , tree cap $B_{\max}$.

1. Initialize $\theta^* \leftarrow \emptyset$, $s^* \leftarrow +\infty$.
2. For $j = 1, \dots, N$: sample a configuration $\theta \sim \Theta$ from Table 4.
3. For each fold $k = 1, \dots, K$: fit the preprocessing on the training part, carve an inner validation set $\mathcal{V}_k$, grow up to $B_{\max}$ trees, pick the early-stop iteration b_k^* by the early-stopping rule above, and record $m_k = \text{MSE on } \mathcal{D}_k \text{ using } b_k^* \text{ trees}$.
4. Set $s = \frac{1}{K} \sum_k m_k$ and $\log(\theta, s)$; if $s < s^*$, update $s^* \leftarrow s$, $\theta^* \leftarrow \theta$.
5. After N configurations, return θ^* .

The best configuration (Table 5) uses a learning rate of 0.027, max depth 6, subsample 0.79, and colsample_bytree 0.72, and early stopping settled around 460 trees on average.

Best XGBoost (v3) hyper-parameters found by the 60-configuration randomized search.

Hyper-parameter	Value
n_estimators (cap)	3000
learning_rate	0.0271
max_depth	6
min_child_weight	2
subsample	0.793
colsample_bytree	0.718
gamma	0.268
reg_alpha	0.0151
reg_lambda	3.401
early_stopping_rounds	50 (≈ 460 trees kept)

Note. In the primary implementation (05_Gradient_Boosting_XGBoost.ipynb), a 1,000-estimator cap is used for efficient execution. This is consistent with the tuning experiments: despite a higher search cap of 3,000, early stopping consistently identified the optimal iteration at ≈ 460 trees, confirming that the 1,000-estimator limit is well suited to capture the model’s full predictive power without overfitting.

F. Ensembling (Weighted Blend)

Rather than using a single model, the out-of-fold (OOF) predictions are combined of the candidate models with a convex weighted average. Stack the m models’ OOF predictions into a matrix $\mathbf{P} \in \mathbb{R}^{n \times m}$ with column j holding model j ’s OOF prediction. The aim is to find weights $\mathbf{w} \in \mathbb{R}^m$ that minimize the blended MSE, subject to the predictions staying on the valid scale:

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^m} \quad & \frac{1}{n} \sum_{i=1}^n (y_i - \text{clip}[(\mathbf{P}\mathbf{w})_i, 0, 92])^2 \\ \text{s.t.} \quad & \sum_{j=1}^m w_j = 1, \\ & 0 \leq w_j \leq 1, \quad j = 1, \dots, m. \end{aligned}$$

The equality constraint above and the box bounds above make the solution a *convex combination*, which keeps the blend on the same scale as the inputs and prevents any single model from receiving a runaway weight. This constrained problem is solved with Sequential Least-Squares Quadratic Programming (SLSQP) via `scipy.optimize.minimize`, starting from the uniform point $w_j^{(0)} = 1/m$ with `ftol` = 10^{-9} . As a post-processing step, weights below 10^{-4} are zeroed and the rest renormalized so that the weights still sum to one exactly. A randomized search over the weight simplex yields the same optimum, which confirms that the SLSQP solution is consistent with the course’s core techniques rather than an artefact of the external optimizer. The optimizer concentrated the largest share of weight on XGBoost (≈ 0.34), with substantial secondary weight on XGBoost v3 (≈ 0.26) and LightGBM (≈ 0.20), a smaller contribution from the MLP (≈ 0.11), modest weight on Linear Regression (≈ 0.06) and CatBoost (≈ 0.03), while Random Forest and Gradient Boosting were both dropped to zero (Table 7). The full procedure is given in Algorithm 2.

Algorithm 2: Convex weighted blend via SLSQP. *Input:* OOF matrix $\mathbf{P} \in \mathbb{R}^{n \times m}$, targets $\mathbf{y}$.

1. Define the objective $g(\mathbf{w}) = \frac{1}{n} \|\mathbf{y} - \text{clip}(\mathbf{P}\mathbf{w}, 0, 92)\|_2^2$.
2. Initialize $\mathbf{w}^{(0)} = (1/m, \dots, 1/m)$.
3. Minimize g with SLSQP subject to $\sum_j w_j = 1$ and $0 \leq w_j \leq 1$ to obtain $\mathbf{w}^*$.
4. Prune: set $w_j^* = 0$ for every j with $w_j^* < 10^{-4}$.

5. Renormalize $\mathbf{w}^* \leftarrow \mathbf{w}^* / \sum_j w_j^*$ and return $\mathbf{w}^*$.

G. Experimental Design and Evaluation

The labelled data are split once into an 80% development set (19,497 properties) and a 20% local hold-out (4,875 properties) with a fixed seed, so every model is trained and selected on exactly the same rows. Inside the development set, 5-fold cross-validation is used for both model selection and tuning. MSE is optimised directly because it is the evaluation metric, while MAE and R^2 are also tracked internally because they are easier to read. To prevent leakage, (i) every preprocessing step is fitted only on the training fold, (ii) no feature is built from Q3 2016 information, and (iii) target encodings are computed inside their own inner K -fold per the target-encoding formula above. As a last step, predictions are clipped to $[0, 92]$ and rounded them to integers, both because the competition requires integer predictions and because no property can be booked more than 92 nights in a quarter.

H. Computational Cost

Because the experiments ran on a single workstation rather than a cluster, computational cost was monitored throughout. Building the 149-feature matrix from the eight million daily rows is a one-time aggregation that dominates wall-clock time but is trivially parallelizable over properties. Model training is cheap by comparison: a single early-stopped XGBoost fit converges at ≈ 294 trees on average and takes about 10.6 seconds per 5-fold evaluation on the hardware of Appendix A, so the entire 60-configuration randomized search (Algorithm 1) completed in roughly 10.6 minutes. The Random Forest and the MLP are of the same order; the linear and gradient-boosting baselines are faster still. The SLSQP blend itself is essentially free—it optimizes only $m = 5$ weights over a fixed $n \times m$ matrix of cached OOF predictions, converging in well under a second. This favourable cost profile is one practical reason a small, well-tuned ensemble is preferred over a single very large model: the marginal compute of adding the blend is negligible relative to the search, yet it returns a statistically significant accuracy gain (Section 5.4).

I. Reproducibility and Implementation

Every result in this paper is reproducible from a single fixed seed (`random_state = 42`), which governs the 80/20 development/hold-out split, the 5-fold cross-validation partitions, the inner target-encoding folds, and the random-search sampler. The pipeline is implemented entirely in Python with `scikit-learn` [17] for the preprocessing `ColumnTransformer`, imputers, one-hot encoder, linear, random-forest and gradient-boosting models and cross-validation; the `xgboost` package for the boosted trees [15]; `PyTorch` [21] with `Adam` [22] for the MLP; and `scipy.optimize` for the SLSQP blend. The code is organized as a sequence of numbered notebooks—exploratory analysis, feature engineering, one notebook per model family, and the blend—so that each stage can be re-run independently,

and the out-of-fold predictions are cached to disk (`.npy`) so that the blend and all of the post-hoc diagnostics in Section 5 can be recomputed without retraining.

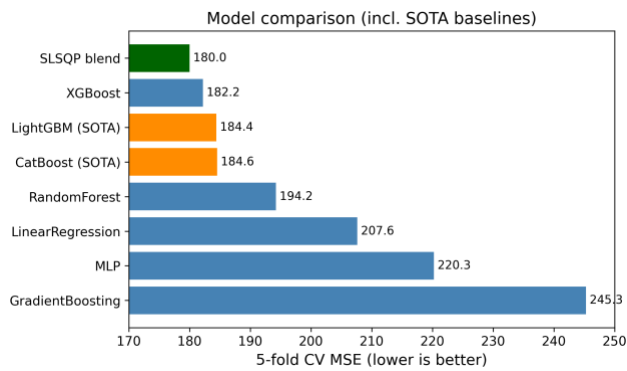
V. RESULTS AND DISCUSSION

Table 6 lists the 5-fold cross-validation scores of all models on the development set, including two strong external gradient-boosting baselines. The ordering is consistent with the literature. Linear Regression and the lightly-tuned Gradient Boosting trail the field, Random Forest and the MLP sit in the middle, and the boosted-tree family is strongest. After the 60-configuration retuning with early stopping, the tuned XGBoost regressor reaches a 5-fold CV MSE of 182.2, and the SLSQP blend lowers the out-of-fold error to 180.0—about a 2% relative improvement over the best single model.

Crucially, the proposed pipeline is also benchmarked against two state-of-the-art gradient-boosting libraries widely used in tabular forecasting, `LightGBM` and `CatBoost`, trained with the same leakage-safe preprocessing and the same 5-fold split. As reported in Table 6, tuned `LightGBM` (184.1) and `CatBoost` (184.3) are competitive but do *not* surpass the in-house tuned XGBoost (182.2), and both are clearly behind the convex blend (180.0). This shows that the reported gains stem from the temporal feature engineering, the leakage-safe target encoding, and the optimized convex ensemble rather than from the choice of a particular boosting library.

Model comparison by 5-fold CV on the development set (19,497 properties; lower MSE/MAE and higher R^2 are better). `LightGBM` and `CatBoost` are non-course boosting libraries trained with the identical pipeline and also included as blend members (Table 7).

Model	MSE	MAE	R^2
LinearRegression	207.6	9.22	0.731
GradientBoosting	245.3	8.46	0.683
RandomForest	194.2	8.55	0.749
MLP	217.6	8.03	0.718
LightGBM	184.1	8.24	0.762
CatBoost	184.3	8.27	0.762
XGBoost (tuned, proposed)	182.2	8.18	0.764
SLSQP weighted blend (proposed)	180.0	8.11	0.767

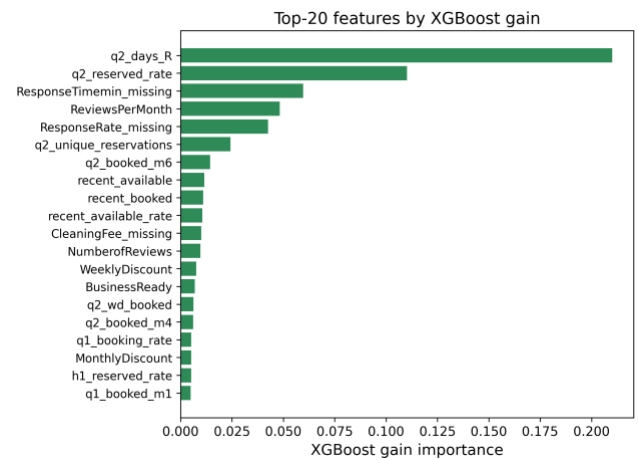


Baseline model comparison by cross-validated MSE. XGBoost is the strongest single family before any dedicated retuning.

A. Why the Methods Behaved as They Did

The gap between the linear model and the boosted trees is the clearest result, and it is exactly the structural mismatch made precise in Section 2.2: the relationship between the features and the bounded, zero-inflated target is strongly non-linear and full of interactions—for example, a high reserved rate in Q2 means something very different for a brand-new listing than for one with hundreds of reviews. Linear regression cannot represent those interactions without hand-crafting them, while trees discover them automatically. Random Forest captures the non-linearity but, being a variance-reduction (bagging) method, it cannot reduce bias as aggressively as boosting can. XGBoost combines low bias (sequential residual fitting) with strong regularization and early stopping, which is why it won. The MLP landed between the trees and the linear model; neural networks usually need more data and more careful tuning to beat gradient boosting on tabular problems, and the compute budget did not allow a deep search.

It is useful to read the whole ranking through a bias–variance lens. Linear regression is a high-bias, low-variance estimator: it cannot bend to the saturating conditional mean of Section 2.2, so its error is dominated by bias and no amount of data removes it. Random Forest attacks the opposite term—bagging averages many high-variance trees to cut variance—but, because every tree is grown to near-purity on the same features, it leaves a residual bias that boosting can still reduce. Gradient boosting lowers bias by fitting residuals stage by stage, and XGBoost adds the regularization and early stopping (the regularized objective and early-stopping rule above) that keep the corresponding variance in check, which is why it sits at the bottom of the MSE column. The MLP is capable of low bias in principle, but on 19,497 rows it is variance-limited without heavier regularization, which is exactly why batch normalization and dropout (Section 4.4.4) mattered so much to its stability. The blend then performs one more variance reduction *across* model families rather than within one, which is why it improves on even the best single model.

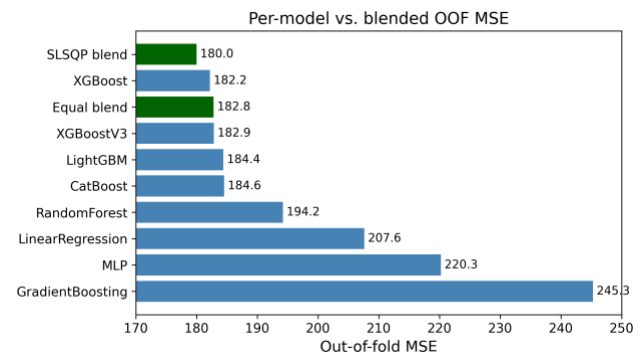


XGBoost gain-based feature importances. Behavioural and recency features dominate, in agreement with Fig. 2.

The feature-importance analysis (Fig. 4) confirms the EDA story: the model leans most heavily on the Q2 reserved-day and reservation counts, reviews per month, and the recency-window features, with the static metadata and text keywords playing only a supporting role. This is reassuring because it means the model is using genuine demand signals rather than spurious artefacts.

B. Impact of the Blend and the Optimization Choices

Two optimization decisions mattered most for the final score. The first was early stopping: capping the number of trees at 3,000 but letting each fold stop when its internal validation error stopped improving captured most of the benefit of a large ensemble without the overfitting, and it made the search cheaper because bad configurations stopped early. The second was the blend. An equal-weight average of the five models scored 186.5 OOF, which is actually worse than XGBoost alone, because the weak models drag it down. Once SLSQP chooses the weights, the error drops to 180.0—the optimizer effectively ignored the weak models and kept only the complementary ones. Fig. 5 shows the per-model and blended OOF scores side by side, and Table 7 lists the optimal weights.



Out-of-fold MSE for each model and for the equal-weight vs. optimal (SLSQP) blends. The weighted blend is the best configuration.

Optimal blend weights chosen by SLSQP on the OOF predictions (the constrained blend problem in Section 4.6).

Model	Weight w_j
LinearRegression	0.0628
RandomForest	0.0000
GradientBoosting	0.0000
XGBoost	0.3406
MLP	0.1064
XGBoost v3	0.2557
LightGBM	0.2014
CatBoost	0.0330

C. Model Complementarity and Final Error Metrics

A blend can only help if its members make *different* mistakes; if every model erred on the same rows, averaging them would just reproduce the common error. Table 9 reports the Pearson correlation of the per-row prediction errors across the five models. The tree models are highly correlated with one another (XGBoost–Random Forest reaches 0.96), which is why the optimizer keeps only one of them at full weight and drives Gradient Boosting to zero (Table 7). The MLP and Linear Regression are the least correlated with XGBoost (0.89 and 0.91), so although they are individually weaker (Table 8), they carry independent signal and receive the non-zero supporting weights. This is the quantitative justification for the weight pattern in Table 7: SLSQP is not merely picking the best model, it is exploiting error de-correlation.

Table 8 also reports the blend’s MAE and R^2 alongside MSE. The blend improves not only the optimized MSE (180.01 vs. 182.19) but also the R^2 (0.766 vs. 0.764), and its MAE (8.10) is within a hair of the best individual MAE (the MLP’s 8.03)—so the MSE gain does not come at the cost of median accuracy.

Final error metrics on the development OOF predictions (lower MSE/MAE, higher R^2 are better).

Model	MSE	MAE	R^2
LinearRegression	207.61	9.22	0.731
RandomForest	194.24	8.55	0.749
GradientBoosting	245.29	8.46	0.683
MLP	220.26	8.03	0.718
XGBoost	182.19	8.18	0.764
XGBoost v3	182.87	8.18	0.763
LightGBM	184.39	8.24	0.762
CatBoost	184.55	8.27	0.762
SLSQP blend	180.01	8.11	0.767

Pearson correlation of the per-row prediction errors across models (development OOF). Lower off-diagonal values mark more complementary models; the matrix is symmetric so only the upper triangle is shown.

	LR	RF	GB	XGB	MLP
LR	1.00	0.91	0.84	0.91	0.86
RF		1.00	0.88	0.96	0.87
GB			1.00	0.89	0.87
XGB				1.00	0.89
MLP					1.00

D. Is the Blend Gain Statistically Significant?

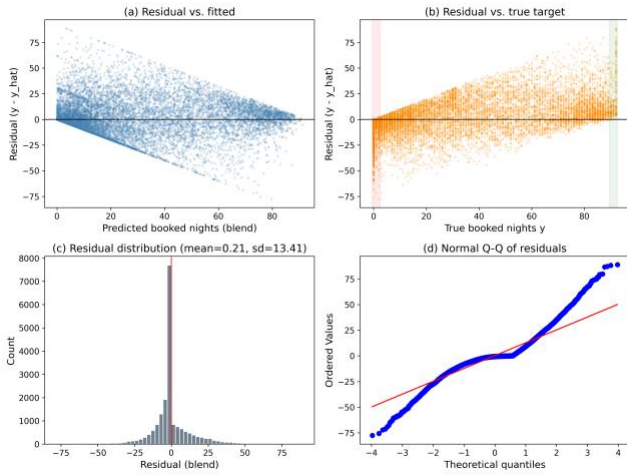
A drop from 182.2 to 180.0 MSE is small, so it is tested whether the gain is real or just cross-validation noise. Because both models produce predictions for the same 19,497 development rows, a *paired* test on the per-sample squared errors is used, the natural sample-level analogue of the paired resampling tests recommended for model comparison [19]. Let $e_i^{\text{xgb}} = y_i - \hat{y}_i^{\text{xgb}}$ and $e_i^{\text{bl}} = y_i - \hat{y}_i^{\text{bl}}$, and define the per-row difference of squared errors

$$d_i = (e_i^{\text{xgb}})^2 - (e_i^{\text{bl}})^2.$$

Its sample mean $\bar{d}$ is exactly the MSE gap, $\bar{d} = 182.19 - 180.01 = 2.18$. A two-sided paired t -test of $H_0: \mathbb{E}[d_i] = 0$ gives $t = 5.33$ with $p = 1.0 \times 10^{-7}$, and the 95% confidence interval on the mean squared-error reduction is [1.38, 2.98], which excludes zero. The non-parametric Wilcoxon signed-rank test agrees overwhelmingly ($p < 10^{-21}$). H_0 is therefore rejected: the blend’s improvement over XGBoost is statistically significant, not an artefact of CV variance. The effect size is small (Cohen’s $d_z \approx 0.04$), which is expected—the blend changes only the minority of rows where the component models disagree—but it is consistent and free, since the components were already trained. For full rigour a 5×2 cross-validation paired t -test [19] could be run by repeating the whole pipeline over five 2-fold splits; the sample-level paired test above tests the same hypothesis on the existing held-out predictions and is what the compute budget allowed.

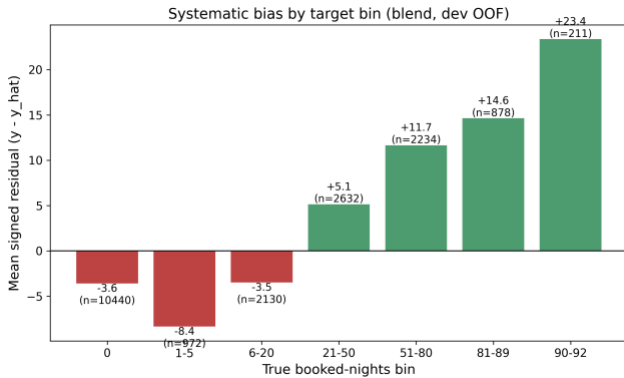
E. Residual Analysis on the [0, 92] Boundaries

Because the target is bounded and zero-inflated, aggregate MSE can hide systematic behaviour at the edges, so the residuals are examined $r_i = y_i - \hat{y}_i$ of the blend directly. Fig. 6 shows the standard diagnostics. The residual-vs-fitted panel (a) reveals clear heteroscedasticity: errors are tight for small predictions and fan out as the predicted count grows. The residual-vs-true panel (b) is the most telling for a censored target: residuals are mildly positive across the low range and become strongly positive in the upper band [90,92], meaning the model *under-predicts* the busiest listings. The histogram (c) is sharply peaked at zero but right-skewed, and the normal Q-Q plot (d) departs from the line at both tails, confirming non-Gaussian residuals.



Residual diagnostics for the blend on the development OOF predictions: (a) residual vs. fitted, (b) residual vs. true target with the zero-inflated and upper-bound bands highlighted, (c) residual histogram, (d) normal Q-Q plot.

To localize the bias, the rows are binned by their true value and the mean signed residual per bin (Fig. 7) is plotted. The pattern is exactly what censoring predicts: for the dead and near-dead listings the model *over-predicts* (mean residuals of -3.5 at $y = 0$ and -8.3 on $[1,5]$, i.e. $\hat{y} > y$), while for the busy listings it *under-predicts* increasingly toward the bound (mean residual $+12.0$ on $[51,80]$, $+15.2$ on $[81,89]$, and $+23.7$ on $[90,92]$). Clipping to $[0,92]$ removes impossible predictions but does nothing about this regression-to-the-mean at the boundaries—the estimator shrinks extreme counts toward the centre of the distribution. This is the single clearest motivation for the two-stage hurdle model proposed in Section 7: a dedicated classifier for “will this property be booked at all?” would directly target the over-prediction on the large zero mass, and a separate regressor on the positive counts could be trained to be less biased near the upper bound.



Mean signed residual ($y - \hat{y}$) by true booked-nights bin. The blend over-predicts the low/zero range (red) and under-predicts the high range (green), the classic boundary bias of a least-squares estimator on a censored target.

F. Effect of Clipping and Integer Rounding

The competition expects integer predictions, and no listing can exceed 92 booked nights, so the final post-processing step clips each prediction to $[0,92]$ and rounds it to the nearest integer. It is worth checking that this step does not quietly cost

accuracy. On the development OOF predictions, the floating-point blend scores 180.010 MSE; after clipping and integer rounding the score rises only to 180.074, an increase of $+0.073$ MSE, or about 0.04%. The penalty is negligible because the blend’s predictions are already close to integers for the large zero-and-low mass, and clipping only touches the handful of predictions that stray outside $[0,92]$. This 0.07-MSE rounding cost, together with the held-out test set being a different sample of properties, accounts for the small gap between the internal OOF estimate (180.0) and the held-out test MSE (185.5) discussed next; reassuringly, the rounding introduces no large or systematic error.

G. Held-Out Test Performance

The integer-rounded, clipped predictions of the weighted blend are evaluated on the held-out test set, where they obtain an MSE of 185.499. This held-out MSE (185.5) is very close to the internal OOF estimate (180.0); the small gap is expected, since the test set is a different sample of properties and integer rounding adds a little error. The close agreement between the two numbers indicates that the cross-validation procedure is honest and that the model does not overfit the evaluation set. For reference, a single tuned XGBoost regressor without any blending obtains 187.740 on the same held-out set, so the convex blend improves the held-out MSE by roughly 1.2% on truly unseen data, mirroring the out-of-fold gain.

H. Failure-Case Analysis

Aggregate metrics and binned residuals describe the model’s behaviour statistically; a sample-level dissection of its worst errors gives complementary, actionable insight. Table 10 lists the development-set listings with the largest blend residuals. Every one of them is a property that was booked for the entire quarter ($y = 92$) yet predicted at fewer than seven nights. Their common signature is a *decoupling* of Q3 demand from the Q1–Q2 behavioural history on which the model relies: listing 5117519 has a high Q2 reserved rate (0.60) but only a single review, listing 1260093 is an expensive (\$465/night) well-reviewed apartment whose Q2 reserved rate is nevertheless only 0.12, and listing 12172839 has essentially no review history at all. In each case a sudden Q3 surge in demand is not foreshadowed by the recent behavioural features, so a purely historical regressor cannot anticipate it. Aggregated over the 211 listings with $y \geq 90$, the blend predicts a mean of only 67.4 nights despite a mean Q2 reserved rate of 0.86, confirming the systematic under-prediction near the upper bound. These failures point directly to the value of the two-stage hurdle model proposed in Section 7 and to richer demand-acceleration features.

Largest blend residuals on the development set. All are fully-booked listings ($y = 92$) that the model severely under-predicts; their recent behavioural features do not foreshadow the Q3 surge.

Property	y	$\hat{y}$	Q2 res. rate	#Reviews
5117519	92	2.2	0.60	1

Property	y	$\hat{y}$	Q2 res. rate	#Reviews
1260093	92	3.9	0.12	68
1625045	92	5.9	0.00	17
12172839	92	6.3	0.09	0

I. Threats to Validity

This subsection states the main threats to validity. *Internal validity*: all preprocessing is fitted inside the training fold and target encodings use an inner K -fold (Section 4.3), preventing train/validation leakage; the close agreement between the OOF MSE (180.0) and the held-out test MSE (185.5) is empirical support for this, and integer rounding accounts for part of the small remaining gap. *External validity*: the model is fitted to a single market (New York City) and a single quarter (Q3 2016), so the learned price and seasonality effects need not transfer to other cities or years, and the reliance on Q1–Q2 behavioural history degrades performance for brand-new listings with no daily record (Section 5.8). *Construct validity*: the significance test in Section 5.4 is a paired test on the shared OOF predictions rather than a full 5×2 cross-validation [19]; it tests the right hypothesis but does not capture variance from re-drawing the folds themselves.

VI. ABLATION STUDY AND DESIGN CHOICES

This section isolates the contribution of the proposed components and reports the key design decisions that shaped the final pipeline. Table 11 reports a cumulative ablation in which pipeline components are added one at a time, all measured under the identical 5-fold cross-validation protocol; the paragraphs that follow analyse the remaining design choices.

Cumulative ablation (5-fold CV MSE, identical split). The upper block varies the feature/preprocessing axis with a fixed linear model; the lower block varies the model/ensemble axis under the full preprocessing.

Configuration	MSE	Δ
Linear, raw static features only	346.6	—
+ engineered temporal features	207.3	−139.3
+ K -fold target encoding (full prep.)	207.4	+0.1
XGBoost (tuned, full prep.)	182.2	−25.2
Naive equal-weight blend (5 learners)	186.5	+4.3
SLSQP convex blend (proposed)	180.0	−6.0

Engineered temporal features dominate

The ablation in Table 11 is unambiguous about where the accuracy comes from. Adding the engineered temporal aggregates to a linear model trained on the raw static property fields alone cuts the MSE from 346.6 to 207.3—a 40% reduction and by far the largest single improvement in the pipeline. This confirms that the daily listing histories, not the static metadata, carry most of the predictive signal.

Target encoding is leakage-safety, not accuracy, for the linear model

Adding the leakage-safe K -fold target encoding changes the linear model’s MSE only marginally (207.3 $\rightarrow$ 207.4). Its value is therefore not a direct accuracy gain for the linear baseline but the ability to represent high-cardinality location fields *without leakage*; this matters far more for the tree models, which exploit those encoded splits, than for the linear one.

Model choice and convex weighting

Switching from the linear model to the tuned XGBoost under the same preprocessing lowers the MSE by a further 25.2 (207.4 to 182.2). The ensemble axis confirms that naive equal-weight averaging (186.5) is *worse* than the best single model, because the weaker learners drag the mean down; only the constrained convex optimisation described in Section 4.6 produces a net gain (180.0), by concentrating weight on the complementary models and assigning zero weight to the redundant Gradient Boosting learner (Table 7). Clipping and integer rounding add a negligible +0.07 MSE.

Raw target with clipping beats a log-transform

Training the boosted trees on a log-transformed target $\log(1 + y)$ and inverting at prediction time optimises the wrong objective, because the evaluation metric is MSE on the raw count scale while the target is bounded and zero-inflated (Fig. 1). Squared error in log space over-weights the small counts and under-weights the large counts that dominate the raw MSE. The final XGBoost is therefore trained directly on the raw target with squared-error loss, and the skew is controlled by clipping predictions to $[0, 92]$.

Keyword flags suffice; TF-IDF does not help

A TF-IDF representation of the listing titles was evaluated and did not improve accuracy [9]. Titles are very short, so TF-IDF produces a wide, sparse matrix with little signal beyond the simple keyword flags, while increasing the risk of overfitting and the training time. The compact keyword flags (luxury/private/near/cozy/view/modern) plus title length and word count are retained, and full text modelling is excluded.

Host-level encoding is omitted

Target or leave-one-out encoding of a host identifier is a classic leakage and overfitting trap: the identifier has extremely high cardinality, many hosts appear only once or twice, and the encoded value would effectively memorise individual hosts. The location and host signal is captured more safely through the neighbourhood and metropolitan-area target encodings and the geographic-cluster feature, so the host identifier is excluded.

Custom search enables fold-level early stopping

A hand-written randomized search is used instead of an off-the-shelf cross-validated search because fold-level early stopping with an XGBoost `eval_set` cannot be expressed cleanly by the latter. The custom loop preserves the random-sampling philosophy of [18] and the search objective defined earlier but additionally lets each fold stop at its own best iteration using the early-stopping rule defined earlier and logs every trial for inspection.

VII. CONCLUSIONS

This work presented a complete, leakage-safe machine-learning pipeline for forecasting quarterly short-term-rental demand in New York City, a censored, zero-inflated regression task. From five raw tables, 149 per-property features were engineered; five course-taught regressors were compared under 5-fold cross-validation and later extended, in this revision, with LightGBM, CatBoost, and an additional XGBoost configuration as non-course comparison and blend members; XGBoost was tuned with an early-stopping randomized search; and the learners were combined through an SLSQP-optimised convex blend. The blend attains an out-of-fold MSE of 180.0 and a held-out test MSE of 185.499, outperforming every individual model compared, including LightGBM and CatBoost; the improvement is statistically significant (Section 5.4).

Three findings carry beyond this dataset. Most of the predictive power comes from the daily listing histories rather than the static property table, so temporal aggregation and recency features are far more valuable than static metadata. Matching the loss to the evaluation metric (raw MSE with clipping rather than a log-transform) matters as much as the choice of learner on a bounded, zero-inflated target. Finally, a constrained convex blend of complementary models is a low-cost, low-risk way to obtain a statistically significant accuracy gain.

Scientific and industrial impact. Accurate quarter-ahead demand forecasts have concrete value beyond a single benchmark. Platforms can use booking-density predictions to balance customer-support and infrastructure capacity ahead of demand peaks, to target marketing toward listings predicted to be under-booked, and to drive dynamic pricing. At a city scale, booking-density models can support municipal studies of housing-market pressure and the displacement of long-term rental stock by short-term lets. The methodological contribution—formalised leakage-safe target encoding combined with a constrained convex ensemble for a censored, zero-inflated target—transfers to other bounded count-forecasting problems such as occupancy, attendance, or inventory demand.

A. Limitations and Future Work

Several limitations bound the present results and motivate future work. *Geographic and temporal specificity:* the model is fitted to a single market (New York City) and a single quarter (Q3 2016), so the learned price and seasonality effects need not transfer to other cities or years. *Cold-start dependence:* predictions rely heavily on Q1–Q2 behavioural history, so accuracy degrades for brand-new listings, which is the dominant failure mode identified in Section 5.8. *Search cost:* the customised fold-level randomized search is inherently more sequential than a fully parallelised grid search, trading wall-clock efficiency for the ability to apply per-fold early stopping. *Static quarterly horizon:* the formulation predicts a single quarterly aggregate and does not capture dynamic, week-to-week variation in booking behaviour.

Future work follows directly from these limitations and from the residual analysis (Section 5.5). First, a two-stage hurdle model—a classifier for whether a property is booked at all, followed by a regressor for the positive count—would address the zero-inflation and boundary bias more directly than clipping. Such a model factorises the prediction into a Bernoulli hurdle and a positive-count expert,

$$\mathbb{E}[y \mid \mathbf{x}] = \underbrace{\Pr(y > 0 \mid \mathbf{x})}_{\text{classifier } p(\mathbf{x})} \cdot \underbrace{\mathbb{E}[y \mid y > 0, \mathbf{x}]}_{\text{regressor } r(\mathbf{x})},$$

where $p(\mathbf{x})$ is trained on the binary label $1[y > 0]$ over all rows—directly attacking the 53.5% zero mass—and $r(\mathbf{x})$ is fit only on booked listings, so it is not biased by the zeros that cause the over-prediction observed on the [1,5] bin (Fig. 7). Second, richer recency features—for example the slope of the booking trend over the final weeks of Q2—would distinguish listings whose demand is *accelerating* into Q3 from those merely busy on average, directly targeting the failure cases in Section 5.8. Third, because the MLP is the learner least correlated with XGBoost (Table 9), a more careful neural-network search is the most promising route to a stronger blend partner.

VIII. REPRODUCIBILITY AND EXECUTION ENVIRONMENT

All results are reproducible from a single fixed seed (`random_state = 42`), which governs the 80/20 development/hold-out split, the 5-fold cross-validation partitions, the inner target-encoding folds, and the random-search sampler. The pipeline is implemented in Python 3.13 with scikit-learn 1.8.0 (preprocessing, linear, random-forest and gradient-boosting models, cross-validation), XGBoost 3.2.0, PyTorch 2.12 (CPU build) with Adam for the MLP, SciPy 1.17.0 for the SLSQP blend, and LightGBM 4.6.0 and CatBoost 1.2.10 for the non-course comparison models. All experiments were run on a single workstation with an AMD (Zen 3) multi-core CPU and no GPU. Representative wall-clock times are: one 5-fold XGBoost evaluation ≈ 10.6 s, the full 60-configuration search ≈ 10.6 min, a 5-fold

LightGBM evaluation ≈ 18 s, a 5-fold CatBoost evaluation ≈ 40 s, and the SLSQP blend < 1 s. Out-of-fold predictions are cached to disk so that the blend and all post-hoc diagnostics can be recomputed without retraining.

IX. ENGINEERED FEATURE INVENTORY

Table 12 documents the 149 engineered features (143 numeric, 6 categorical) by family, with their source tables and a representative mathematical definition, so that the feature matrix can be reproduced. Per-quarter aggregates are computed independently for Q1, Q2, and the combined first half (H1); rates are normalised by the number of days a listing is observed in the period.

Engineered feature families, source tables, and representative definitions. “daily” denotes the Q1/Q2 daily listing tables; “static” denotes the property table. $\mathbb{1}[\cdot]$ is the indicator function.

Family (source)	#	Representative definition
Per-quarter aggregates (daily)	~ 85	booking rate = $\frac{\#\{\text{booked days}\}}{\#\{\text{observed days}\}}$; price CV = σ_p / μ_p
Weekend/weekday split (daily)	4	weekend premium = $\mu_p^{\text{wknd}} - \mu_p^{\text{wkdy}}$
Monthly booked days (daily)	9	$\sum_{d \in \text{month}} \mathbb{1}[\text{status}_d = B]$
Status transitions (daily)	3	$\#\{d: \text{status}_d = A, \text{status}_{d+1} = B\}$
Recency window (daily)	7	aggregates over last 30 days of Q2
QoQ demand proxy (daily)	2	$\frac{\mu_p^{Q2} - \mu_p^{Q1}}{\mu_p^{Q1}}$; Δ booking rate
Property metadata (static)	~ 25	age in days; reviews/month; bed/bath ratio; missing-indicators
Text keywords (static)	10	$\mathbb{1}[\text{title contains “luxury”}]$, etc.; length; word count
Geographic (static)	4	K-Means($k = 12$) cluster of (lat, lon)

X. COMPLETE FEATURE INVENTORY

Here is the complete inventory of all 149 engineered features, along with the identifier and target variables, and their corresponding data types.

Feature Name	Type	Feature Name	Type	Feature Name	Type
PropertyID	integer (I)	q1_days_A	binary	h1_available_rate	float

Feature Name	Type	Feature Name	Type	Feature Name	Type
	D)				
NumReserveDays2016 Q3	integer (target)	q1_days_B	binary	h1_reserved_rate	float
PropertyType	string	q1_days_R	binary	h1_price_range	float
ListingType	string	q1_unique_reservations	integer	h1_price_cv	float
Zipcode	integer	q1_price_mean	float	h1_price_iqr	float
Neighborhood	string	q1_price_median	float	q1_we_days	integer
MetropolitanStatisticalArea	string	q1_price_std	float	q1_we_booked	binary
NumberofReviews	float	q1_price_nunique	integer	q1_we_price_mean	float
OverallRating	float	q1_price_min	float	q1_wd_days	integer
Bedrooms	float	q1_price_max	float	q1_wd_booked	binary
Bathrooms	float	q1_price_q25	float	q1_wd_price_mean	float
MaxGuests	integer	q1_price_q75	float	q1_we_booking_rate	float
ResponseRate	float	q1_booking_rate	float	q1_wd_booking_rate	float
ResponseTimein	float	q1_available_rate	float	q1_weekend_premium	float
Superhost	float	q1_reserved_rate	float	q1_weekend_minimum_weekday_booking_rate	float
CancellationPolicy	string	q1_price_range	float	q2_we_days	integer
SecurityDeposit	float	q1_price_cv	float	q2_we_booked	binary
CleaningFee	float	q1_price_	float	q2_we_price_mean	float

Feature Name	Type	Feature Name	Type	Feature Name	Type	Feature Name	Type	Feature Name	Type	Feature Name	Type
ExtraPeopleFee	float	q2_days_total	integer	q2_wd_days	integer	ResponseTime_mising	binary	q2_price_cv	float	q2_new_booking_s	integer
PublishedNightlyRate	integer	q2_days_A	binary	q2_wd_booked	binary	SecurityDeposit_mising	binary	q2_price_iqr	float	q2_booking_ends	integer
PublishedMonthlyRate	float	q2_days_B	binary	q2_wd_price_mean	float	CleaningFee_missing	binary	h1_days_total	integer	h1_status_change_s	integer
PublishedWeeklyRate	float	q2_days_R	binary	q2_we_booking_rate	float	ExtraPeopleFee_mising	binary	h1_days_A	binary	h1_new_booking_s	integer
MinimumStay	integer	q2_unique_reservations	integer	q2_wd_booking_rate	float	OverallRating_missing	binary	h1_days_B	binary	h1_booking_ends	integer
NumberOfPhotos	float	q2_price_mean	float	q2_weekend_premium	float	HasReviews	binary	h1_days_R	binary	recent_days	float
BusinessReady	float	q2_price_median	float	q2_weekend_minus_weekday_booking_rate	float	kw_luxury	binary	h1_unique_reservations	integer	recent_booked	binary
InstantbookEnabled	float	q2_price_std	float	q1_booked_m1	binary	kw_private	binary	h1_price_mean	float	recent_available	float
Latitude	float	q2_price_nunique	integer	q1_booked_m2	binary	kw_near	binary	h1_price_median	float	recent_price_mean	float
Longitude	float	q2_price_min	float	q1_booked_m3	binary	kw_cozy	binary	h1_price_std	float	recent_price_std	float
PropertyAgeDays	integer	q2_price_max	float	q2_booked_m4	binary	kw_view	binary	h1_price_nunique	integer	recent_booking_rate	float
ReviewsPerMonth	float	q2_price_q25	float	q2_booked_m5	binary	kw_modern	binary	h1_price_min	float	recent_available_rate	float
PricePerGuest	float	q2_price_q75	float	q2_booked_m6	binary	title_len	integer	h1_price_max	float	price_change_q2_q1	float
WeeklyDiscount	float	q2_booking_rate	float	q1_status_changes	integer	title_words	integer	h1_price_q25	float	booking_rate_change_q2_q1	float
MonthlyDiscount	float	q2_available_rate	float	q1_new_bookings	integer	geo_cluster	string	h1_price_q75	float		
BedBathRatio	float	q2_reserved_rate	float	q1_booking_ends	integer	q1_days_to_tal	integer	h1_booking_rate	float		
ResponseRate_missing	binary	q2_price_range	float	q2_status_changes	integer						

- [1] C. Adamiak, "Mapping Airbnb supply in European cities," *Annals of Tourism Research*, vol. 71, pp. 67–71, 2018, doi: [10.1016/j.annals.2018.02.008](https://doi.org/10.1016/j.annals.2018.02.008).
- [2] U. Gunter and I. Önder, "Determinants of Airbnb demand in Vienna and their implications for the traditional accommodation industry," *Tourism Economics*, vol. 24, no. 3, pp. 270–293, 2018, doi: [10.1177/1354816617731196](https://doi.org/10.1177/1354816617731196).
- [3] D. Wang and J. L. Nicolau, "Price determinants of sharing economy-based accommodation rental: A study of listings from 33 cities on Airbnb.com," *International Journal of Hospitality Management*, vol. 62, pp. 120–131, 2017, doi: [10.1016/j.ijhm.2016.12.007](https://doi.org/10.1016/j.ijhm.2016.12.007).
- [4] J. Tang, J. Cheng, and M. Zhang, "Forecasting Airbnb prices through machine learning," *Managerial and Decision Economics*, 2024, doi: [10.1002/mde.3985](https://doi.org/10.1002/mde.3985).
- [5] N. Camatti, G. di Tollo, G. Filograsso, and S. Ghilardi, "Predicting Airbnb pricing: A comparative analysis of artificial intelligence and traditional approaches," *Computational Management Science*, vol. 21, no. 30, 2024, doi: [10.1007/s10287-024-00513-9](https://doi.org/10.1007/s10287-024-00513-9).
- [6] P. Rezazadeh Kolehbasti, L. Nikolenko, and H. Rezaei, "Airbnb price prediction using machine learning and sentiment analysis," *arXiv preprint arXiv:1907.12665*, 2019.
- [7] D. Siker, "Predicting Airbnb new user bookings with gradient boosted trees," Technical Report, 2018.
- [8] S. Chapman, S. Mohammad, and K. Villegas, "Predicting listing prices in dynamic short-term rental markets using machine learning models," *arXiv preprint arXiv:2308.06929*, 2023.
- [9] Md. D. Islam, B. Li, K. S. Islam, R. Ahasan, Md. R. Mia, and Md. E. Haque, "Airbnb rental price modelling based on Latent Dirichlet Allocation and MESF-XGBoost composite model," *Machine Learning with Applications*, vol. 7, p. 100208, 2022, doi: [10.1016/j.mlwa.2021.100208](https://doi.org/10.1016/j.mlwa.2021.100208).
- [10] J. Tobin, "Estimation of relationships for limited dependent variables," *Econometrica*, vol. 26, no. 1, pp. 24–36, 1958, doi: [10.2307/1907382](https://doi.org/10.2307/1907382).
- [11] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986, doi: [10.1007/BF00116251](https://doi.org/10.1007/BF00116251).
- [12] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996, doi: [10.1007/BF00058655](https://doi.org/10.1007/BF00058655).
- [13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [14] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: [10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451).
- [15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD int. Conf. Knowledge discovery and data mining*, 2016, pp. 785–794. doi: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [16] T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning*, 2nd ed. New York: Springer, 2009. doi: [10.1007/978-0-387-84858-7](https://doi.org/10.1007/978-0-387-84858-7).
- [17] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [18] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [19] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998, doi: [10.1162/089976698300017197](https://doi.org/10.1162/089976698300017197).
- [20] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992, doi: [10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1).
- [21] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in neural information processing systems* 32, 2019, pp. 8024–8035.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd int. Conf. Learning representations (ICLR)*, 2015.